EEC · COMPUTER VISION · MCE / MSc CS

Chapter 2

Deep Learning for Computer Vision

From convolutions to trained, fine-tuned vision models

Compiled by: Niraj Pahari, PhD

CHAPTER ROADMAP

Four Building Blocks of This Chapter

2.1

Convolutional Neural Networks

How a network learns to see — kernels, feature maps, pooling, classic architectures.

2.2

Training & Optimization

How a network actually learns — loss, gradients, backpropagation, optimizers.

2.3

Regularization & Tuning

How we stop a network from memorizing instead of learning — dropout, augmentation, search.

2.4

Transfer Learning & Domain Adaptation

How we reuse what one network already learned to solve a new, smaller problem.

Why Not Just Use a Fully-Connected Network?

- A plain (fully-connected) network treats an image as one giant flat list of pixels.
- Every pixel connects to every neuron in the next layer — spatial structure (what's near what) is thrown away.
- The number of weights explodes with image size, so the network overfits fast and trains slowly.
- CNNs fix this with two ideas: local connections and shared weights (the same small filter slides across the whole image).

FULLY-CONNECTED LAYER

~150,000,000

weights to connect one 224x224 RGB image to just 1,000 neurons

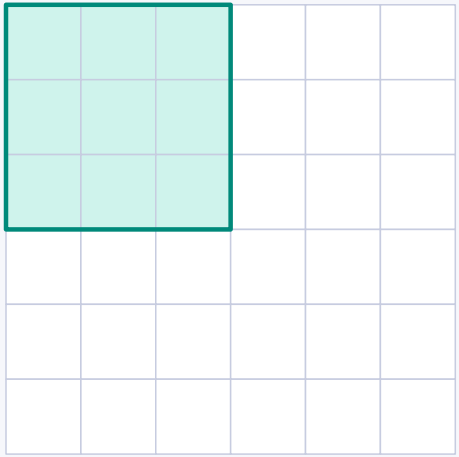
CONVOLUTIONAL LAYER

~900

weights for a 3x3 filter with 32 output channels — reused at every image position

Convolution: Sliding a Small Filter Over an Image

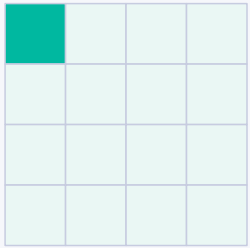
INPUT IMAGE (6x6)



FILTER / KERNEL (3x3)

1	0	-1
1	0	-1
1	0	-1

OUTPUT FEATURE MAP (4x4)



How it works

- The teal 3×3 box is the filter's current position on the image (its receptive field).
- Multiply each overlapping pair of numbers and sum them → one output value.
- Slide the filter one step right (stride), repeat, until the whole image is covered.
- Different filters detect different patterns: edges, textures, colors.

Computing One Output Value, Step by Step

INPUT (5x5)

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

KERNEL (3x3)

1	0	1
0	1	0
1	0	1

Elementwise multiply, then sum

```
(1x1)+(1x0)+(1x1)
+(0x0)+(1x1)+(1x0)
+(0x1)+(0x0)+(1x1)
```

```
= 1 + 0 + 1 + 0 + 1 + 0
+ 0 + 0 + 1
```

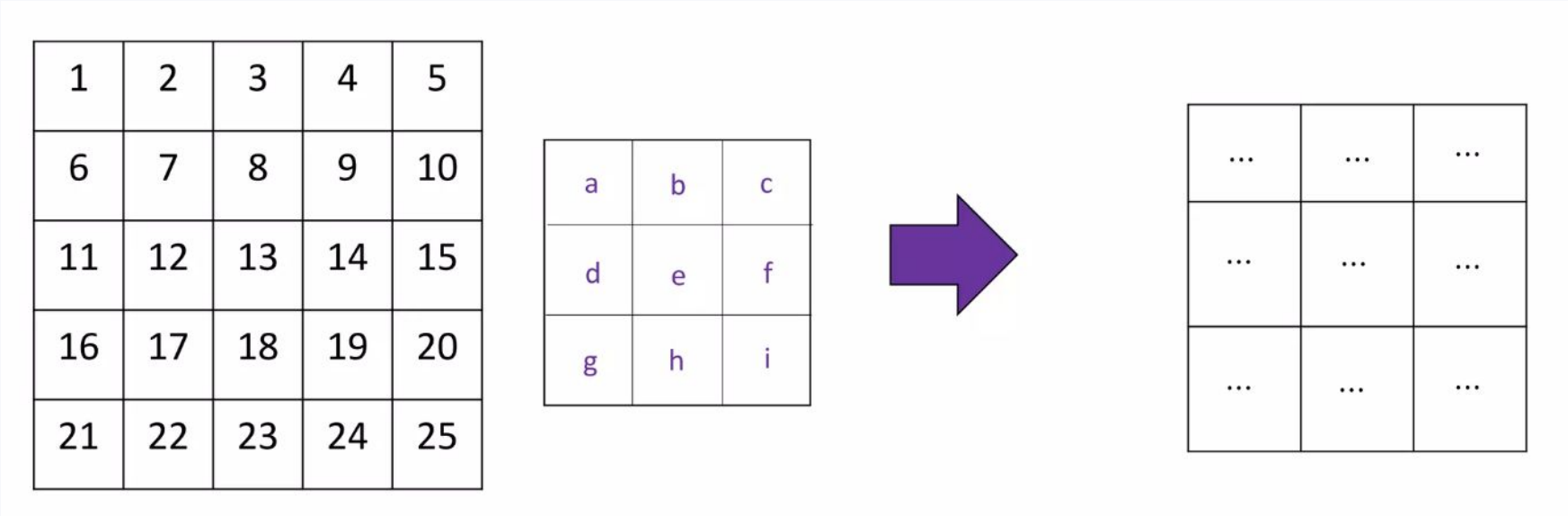
= 4

This single number becomes one cell of the output feature map. Slide the kernel one step right and repeat for the next cell.

Input

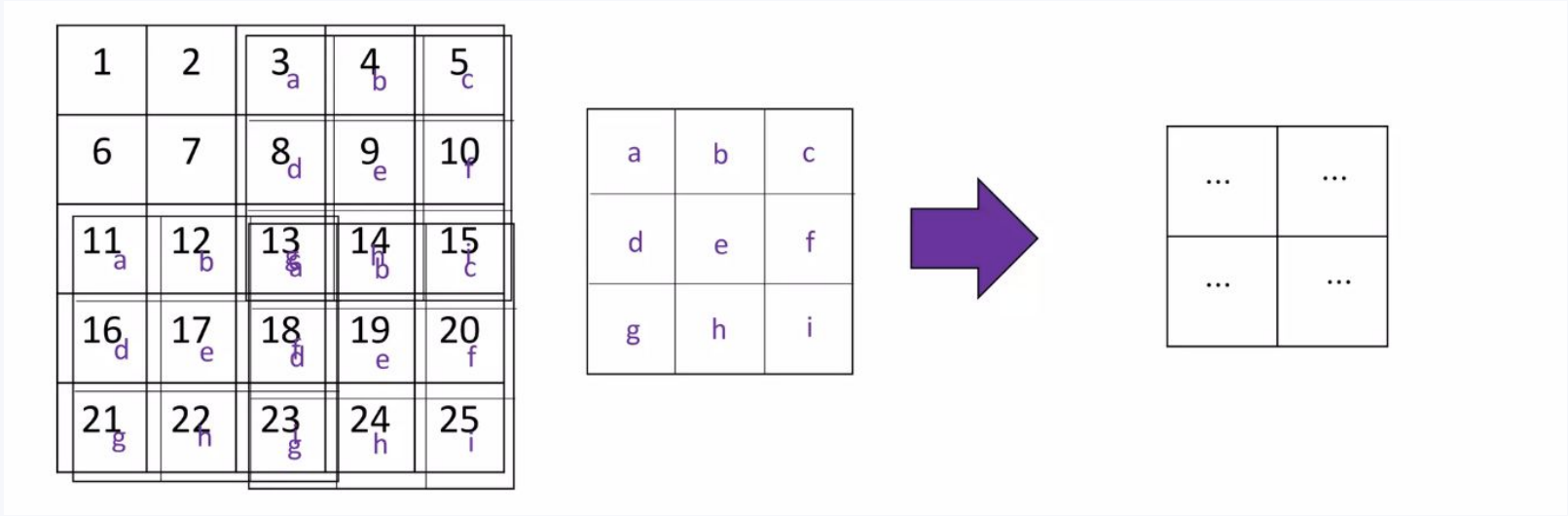
Size of convoluted array

- We can see, the size of layer becomes smaller after convolution.



Size of convoluted array

- If we skip some block, the convolved layer also gets smaller.
- This is called ‘stride’.



Padding

- Adding extra pixels (usually zeros) around the border of an input image/feature map before convolution.

- **Purpose**:
  - Prevents shrinking of feature maps after each convolution.
  - Preserves information at the edges/corners of the image (otherwise edge pixels are used less often than central pixels).
  - Allows building deeper networks without the output size vanishing too quickly.

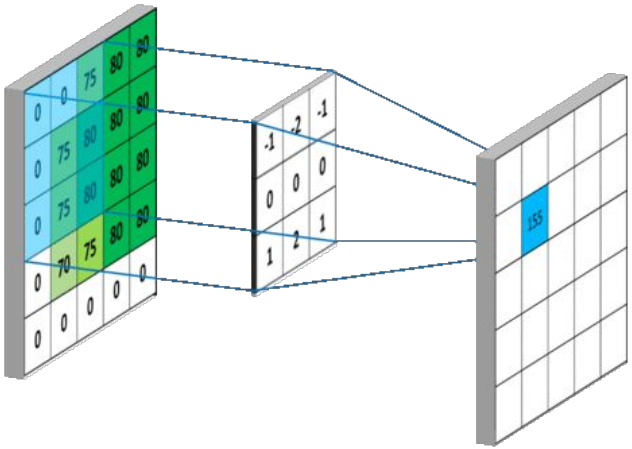
Padding

- **Types**:
  - **Valid padding (no padding)**: No extra pixels added; output size shrinks after convolution.
  - **Same padding**: Padding added so that output size equals input size (when stride = 1).

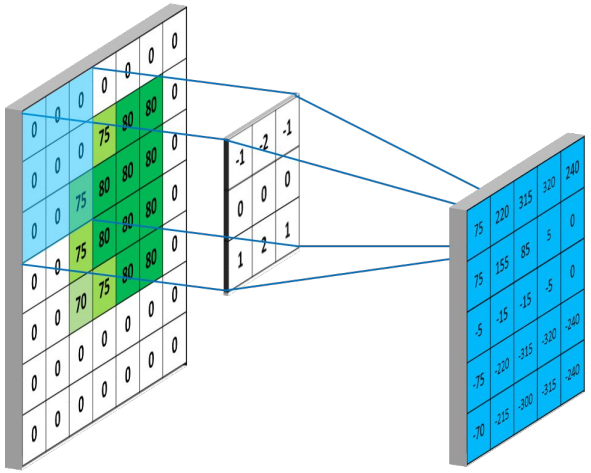
Stride

- The number of pixels by which the filter/kernel moves across the input at each step.

- **Purpose**:
  - Controls how much the filter "jumps" — affects output size and computation.
  - Larger strides → smaller output size, less computation, less detail captured.
  - Smaller strides (stride = 1) → larger output, more detail, more overlap between receptive fields.



The kernel is moved over the image performing the convolution as it goes.



Zero-padding is used so that the resulting image doesn't shrink.

Pooling

- A downsampling operation that reduces the spatial dimensions (height & width) of feature maps while retaining the most important information.

- **Purpose**:
  - Reduces computational load and number of parameters in the network.
  - Controls overfitting by providing an abstracted, summarized representation.
  - Makes the network more robust to small translations/distortions in the input (translation invariance).
  - Helps capture dominant features while discarding less useful detail.

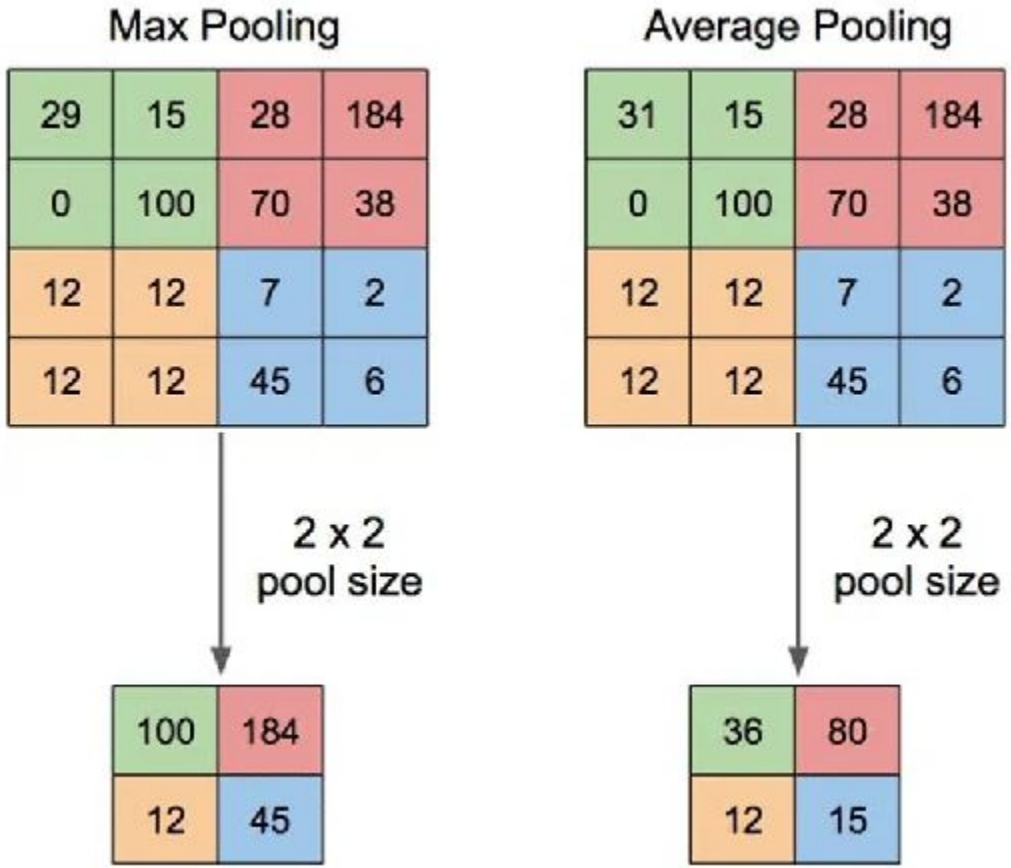
Pooling

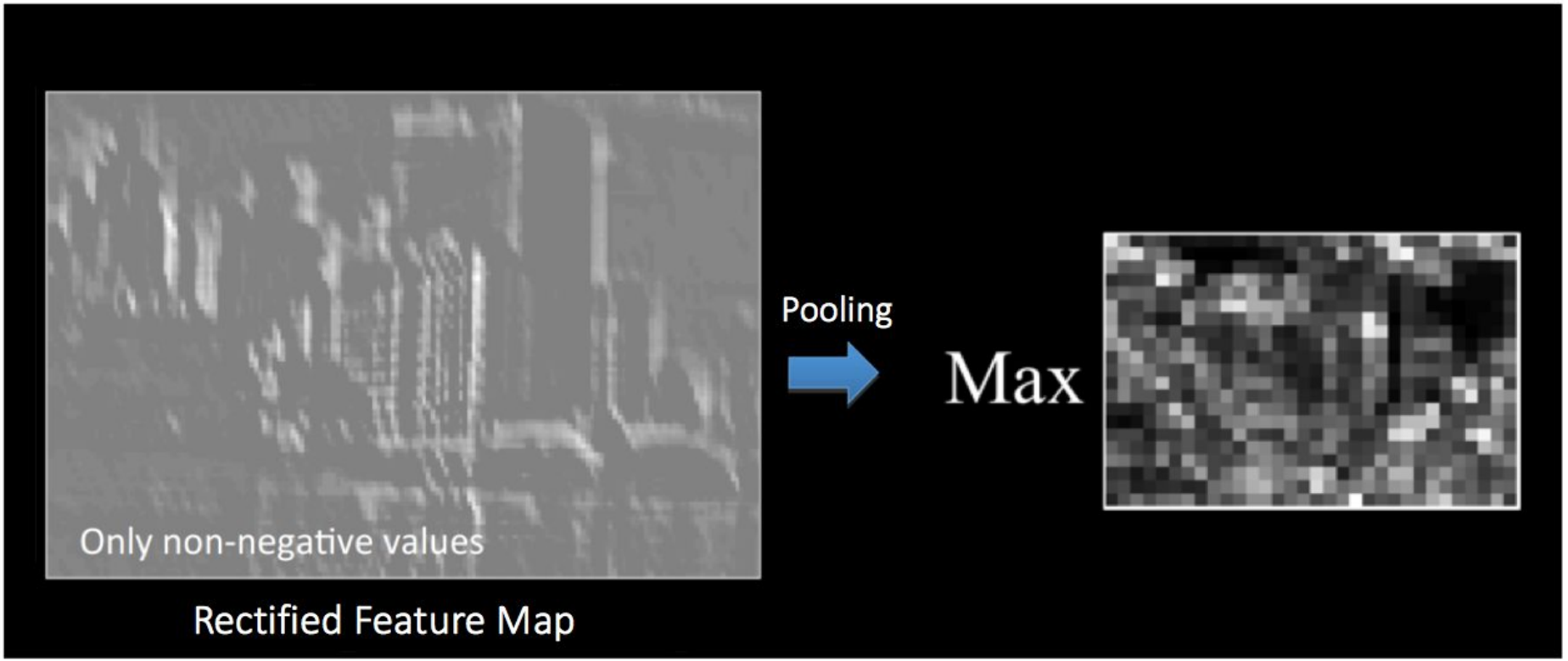
- **Types**:
  - **Max Pooling:**
    - Takes the maximum value from each patch of the feature map.
    - Most commonly used; preserves the strongest activations (sharp features, edges).
  - **Average Pooling:**
    - Takes the average value from each patch.
    - Smooths the feature map; less commonly used in hidden layers but sometimes used at the end (Global Average Pooling).
  - **Global Average Pooling (GAP):**
    - Averages the entire feature map into a single value per channel.
    - Often used before the final classification layer to reduce parameters drastically.

Pooling

- **Key Parameters**
  - **Pool size (filter size)**
    - E.g., 2×2 — size of the window the pooling operation slides over.
  - **Stride**
    - How far the pooling window moves each step (often equal to pool size, so regions don't overlap).
  - **Padding**
    - Rarely used in pooling, but can be applied similarly to convolution.

- **Example**: A **4×4** feature map with **2×2** max pooling and stride **2** → output is **2×2**, where each output value is the max of a non-overlapping 2×2 region.


Max Pooling
29
15
28
184
0
100
70
38
12
12
7
2
12
12
45
6
2 x 2
pool size
100
184
12
45
Average Pooling
31
15
28
184
0
100
70
38
12
12
7
2
12
12
45
6
2 x 2
pool size
36
80
12
15


Only non-negative values
Rectified Feature Map
Pooling
Max

Controlling Output Size: Padding, Stride & Pooling

Padding

ADD A BORDER OF ZEROS

Without padding, the output shrinks every layer and edge pixels get used less. 'Same' padding adds a zero border so output size = input size.

Stride

STEP SIZE OF THE SLIDE

Stride 1 moves the kernel one pixel at a time (dense output). Stride 2 skips a pixel each step, halving the output size — a cheap way to downsample.

Pooling

SHRINK, KEEP THE SIGNAL

Max-pooling keeps the strongest activation in each small region; average-pooling keeps the mean. Both reduce size and add small-shift tolerance.

1	3	2	4
5	6	1	2
1	2	0	1
3	1	2	8

max of each 2x2 -> 6, 4, 3, 8

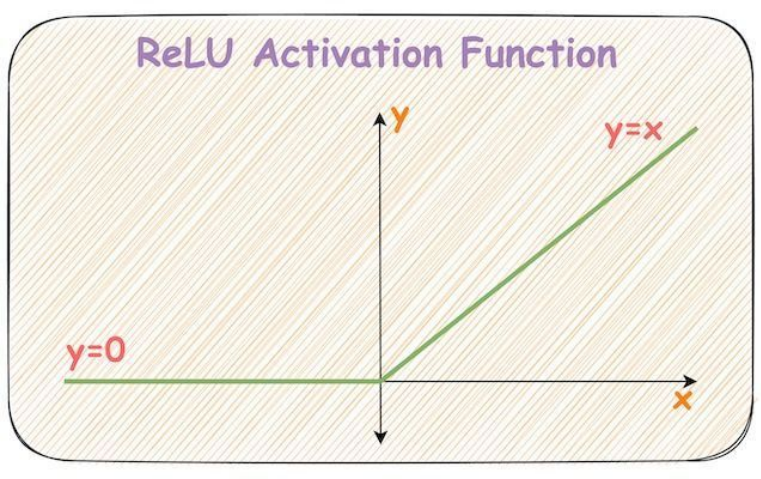
Activation Function

- A mathematical function applied to the output of a neuron (after the weighted sum) to introduce non-linearity into the network.

- **Why needed**:
  - Without activation functions, a CNN (no matter how many layers) would behave like a single linear transformation — unable to learn complex patterns like edges, textures, or shapes.

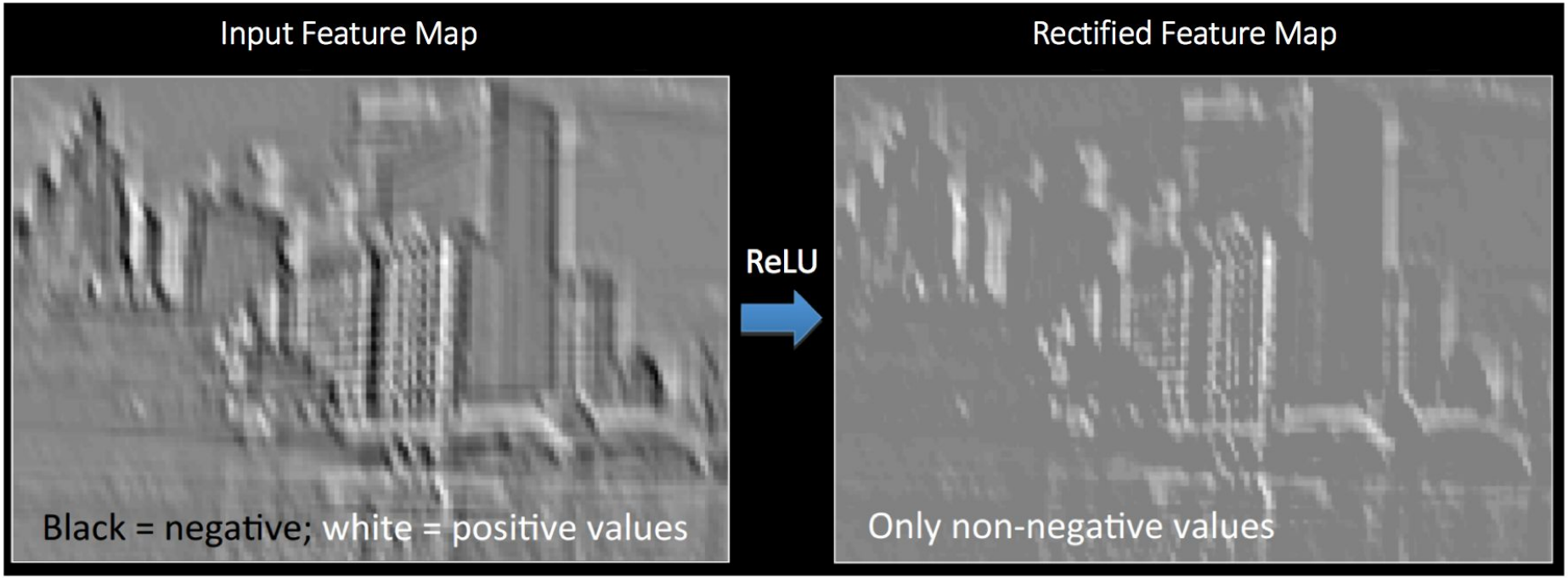
- **Where activation functions are used in a CNN?**
  - After convolution layers (most common use).
  - After fully connected layers.
  - At the output layer (final classification/regression).

Why ReLU? Adding Non-Linearity Between Layers



ReLU: $f(x) = \max(0, x)$

- Passes positive values through unchanged; zeroes out negatives.
- Cheap to compute and keeps gradients strong for positive inputs, unlike sigmoid/tanh which 'flatten out' (saturate) at the extremes.
- Saturating activations make gradients shrink toward zero in deep networks — this is the vanishing-gradient problem.
- Without ANY non-linear activation, stacking conv layers collapses mathematically into a single linear layer — depth would add no power.


Input Feature Map
Rectified Feature Map
ReLU
Black = negative; white = positive values
Only non-negative values

Feedforward Neural Network (Fully Connected Layer)

- The final part of a CNN where the flattened, high-level features extracted by convolution and pooling layers are passed through one or more fully connected (dense) layers to perform classification or regression.

- **Why "feedforward"**
  - Information flows in one direction — from input to output — with no loops or feedback connections (as opposed to RNNs).

- **Position in a CNN architecture**
  - **Typical pipeline:**
  - Input image → Convolution layers → Activation (ReLU) → Pooling layers → (repeat) → Flatten → Fully Connected (FFN) layers → Output layer (Softmax/Sigmoid)

- The FFN sits at the end of the network, after feature extraction is complete.

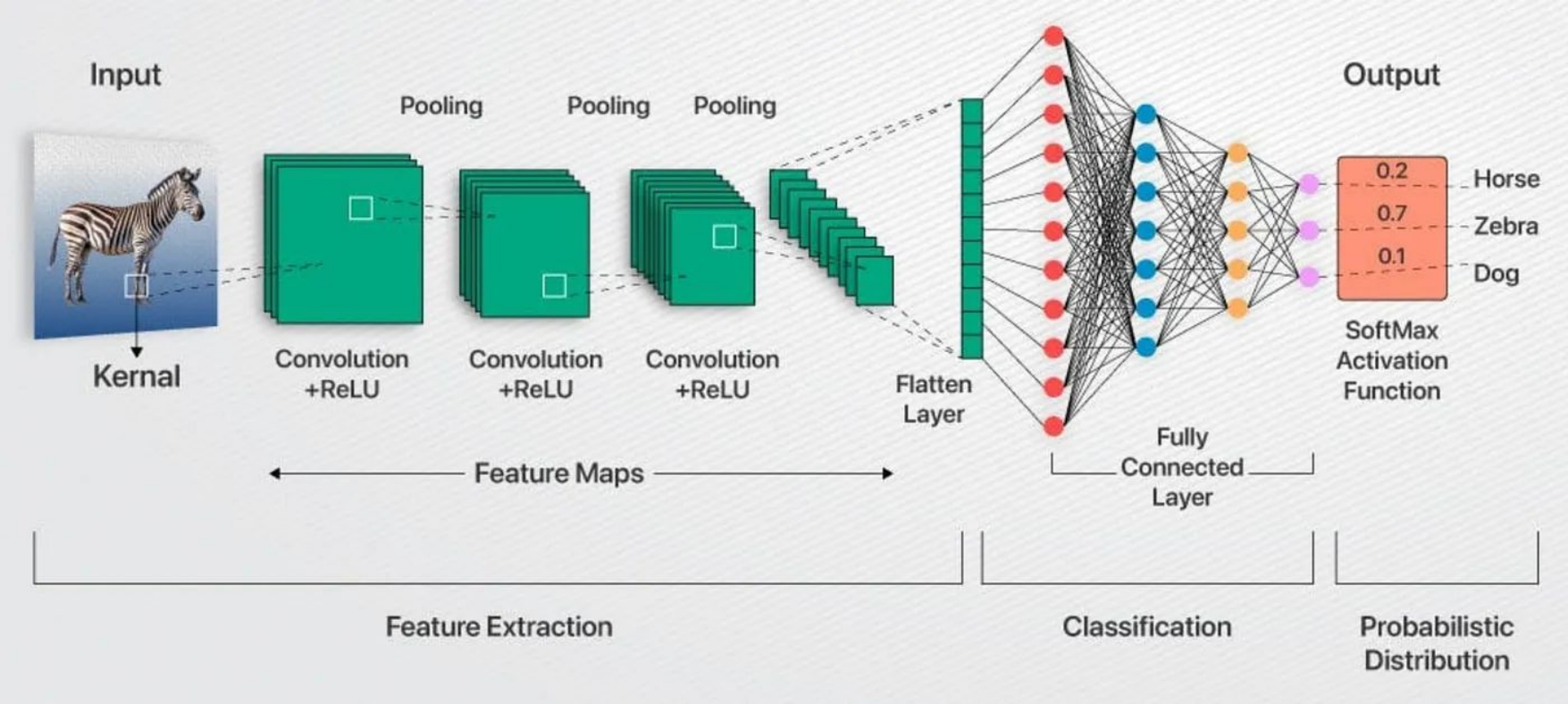
CNN - Key Steps

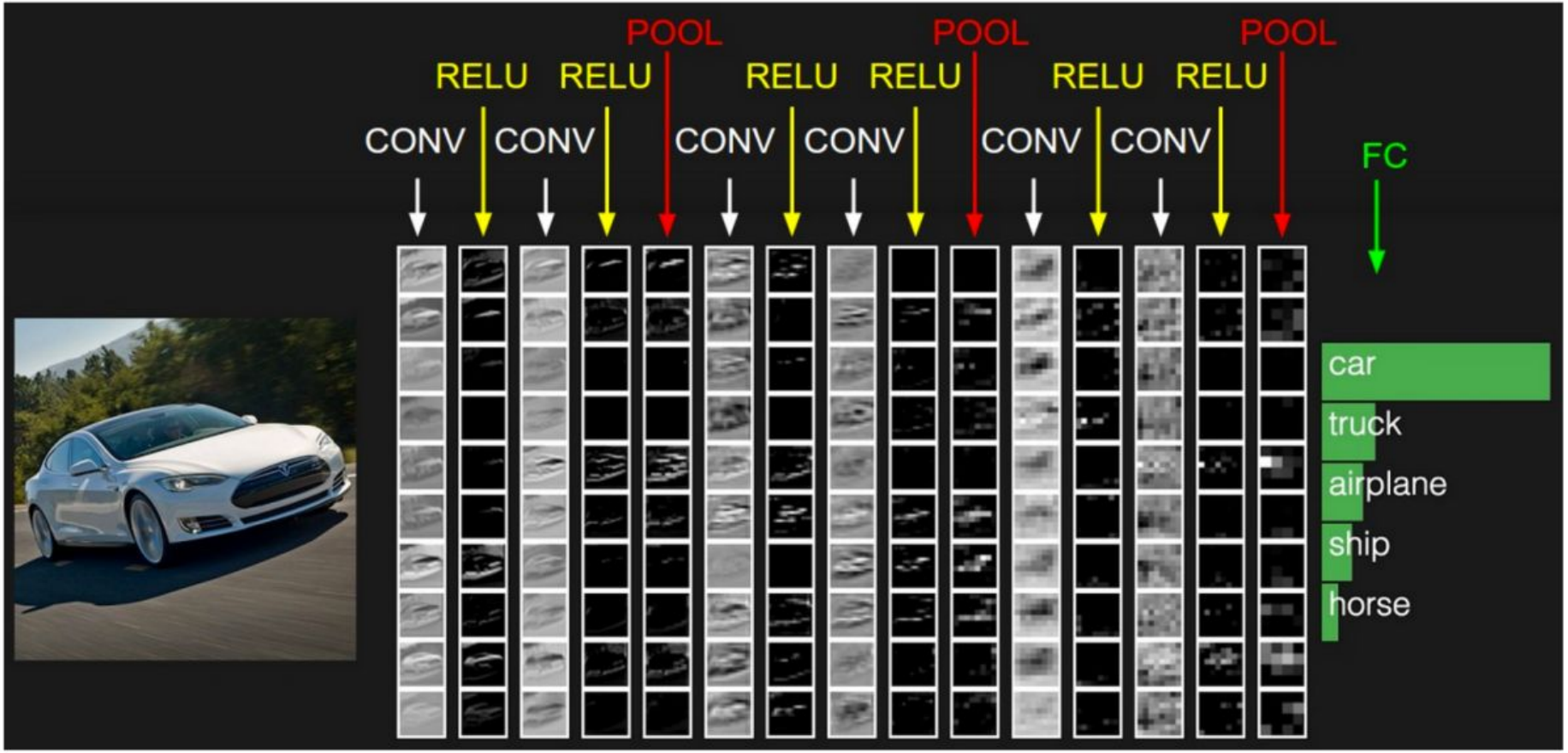
- **Flattening**
  - Converts the 2D/3D feature maps (output of conv/pooling layers) into a 1D vector.
  - Example: A 7×7×64 feature map → flattened into a vector of 3136 values.

- **Fully Connected Layers**
  - Every neuron in one layer connects to every neuron in the next (dense connections).
  - Each connection has a learnable weight; each neuron has a bias.
  - Computes: output = activation(W·x + b)

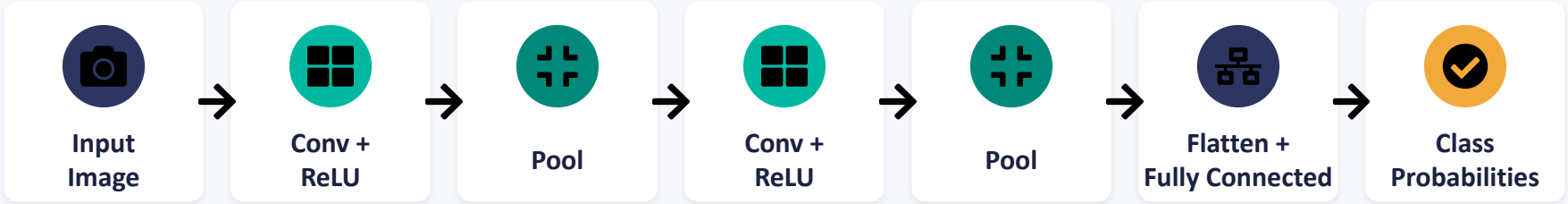
- **Activation Functions**
  - ReLU is commonly used in hidden FC layers.
  - Softmax (multi-class) or Sigmoid (binary) used in the final output layer to produce probabilities.

CNN


Input
Pooling
Pooling
Pooling
Output
0.2
0.7
0.1
Horse
Zebra
Dog
Kernal
Convolution +ReLU
Convolution +ReLU
Convolution +ReLU
Flatten Layer
SoftMax Activation Function
Feature Maps
Fully Connected Layer
Feature Extraction
Classification
Probabilistic Distribution


POOL
POOL
POOL
RELU
RELU
RELU
RELU
RELU
RELU
CONV
CONV
CONV
CONV
CONV
CONV
FC
car
truck
airplane
ship
horse

Putting It Together: A Full CNN Pipeline

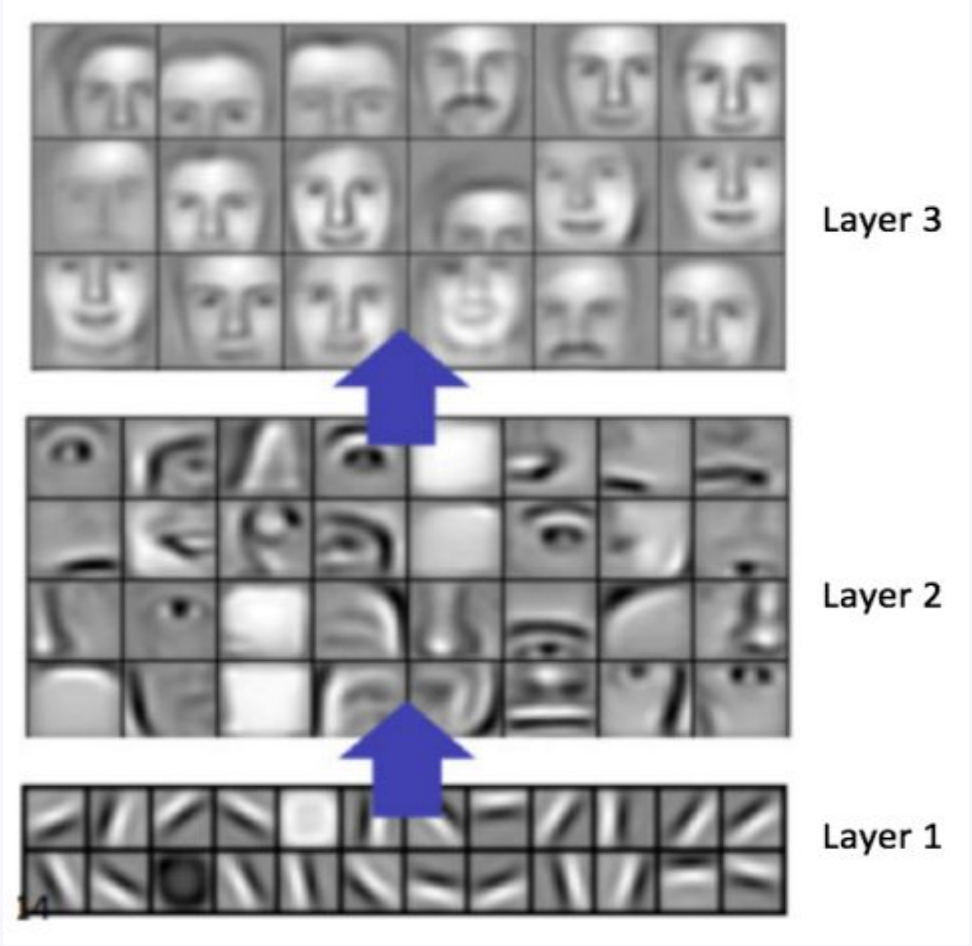


Spatial size shrinks (224 -> 112 -> 56 ...) while depth (channel count) grows — the network trades spatial detail for richer, more abstract features.

Early layers learn edges & colors

Middle layers learn textures & parts

Late layers learn whole objects



Learned features from CNNS

Visualization

DEMO

Milestones: How CNN Architectures Evolved

Architecture	Year	Depth	Key Innovation
LeNet-5	1998	7 layers	First practical CNN; digit recognition (MNIST)
AlexNet	2012	8 layers	ReLU + dropout + GPU training; won ImageNet by a huge margin
VGG-16/19	2014	16-19 layers	Proved 'just stack small 3x3 filters, go deeper' works
ResNet	2015	Up to 152 layers	Skip / residual connections solve the vanishing-gradient limit on depth

The big trend: deeper networks learn richer features — but only once tricks like ReLU, dropout, batch norm, and residual connections solved the training problems that depth introduces.

```python
import torch.nn as nn
import torch.nn.functional as F

class Net(nn.Module):

    def __init__(self, n_classes):
        super(Net, self).__init__()

        # 1 input image channel (grayscale), 32 output channels/feature maps
        # 5x5 square convolution kernel
        self.conv1 = nn.Conv2d(1, 32, 5)

        # maxpool layer
        # pool with kernel_size=2, stride=2
        self.pool = nn.MaxPool2d(2, 2)

        # fully-connected layer
        # 32*4 input size to account for the downsampled image size after pooling
        # num_classes outputs (for n_classes of image data)
        self.fc1 = nn.Linear(32*4, n_classes)
```

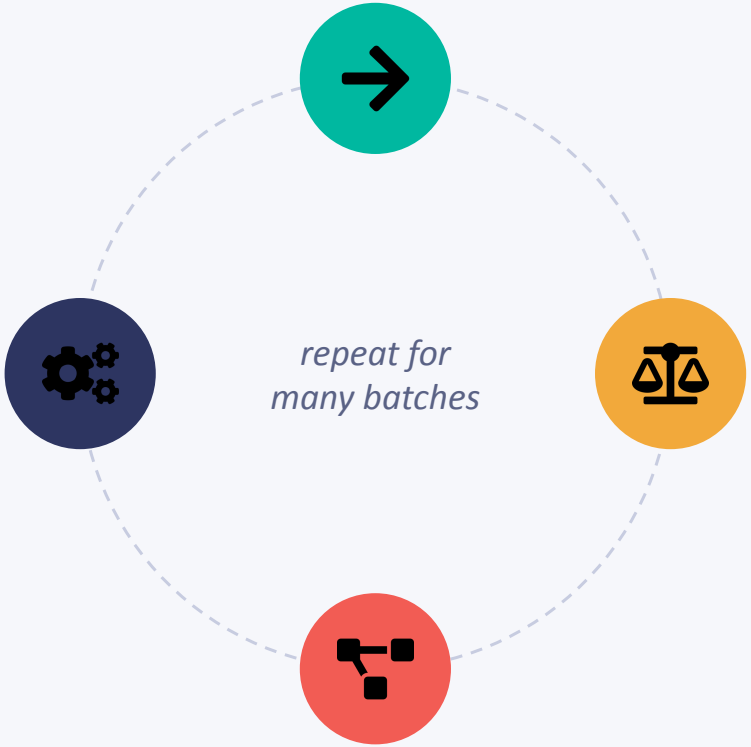
```
    # define the feedforward behavior
    def forward(self, x):
        # one conv/relu + pool layers
        x = self.pool(F.relu(self.conv1(x)))

        # prep for linear layer by flattening the feature maps into feature vectors
        x = x.view(x.size(0), -1)
        # linear layer
        x = F.relu(self.fc1(x))

        # final output
        return x

# instantiate and print your Net
n_classes = 20 # example number of classes
net = Net(n_classes)
print(net)

```

SECTION 2.2

Training & Optimization

How a network actually learns: loss, gradients, backpropagation, and optimizers.

The Training Loop, One Cycle at a Time



1. **Forward Pass**
   Push a batch of images through the network to get predictions.
2. **Compute Loss**
   Compare predictions to true labels with a loss function.
3. **Backward Pass**
   Backpropagation computes how much each weight contributed to the error.
4. **Update Weights**
   The optimizer nudges every weight slightly to reduce the loss.

Measuring Error, Then Walking Downhill

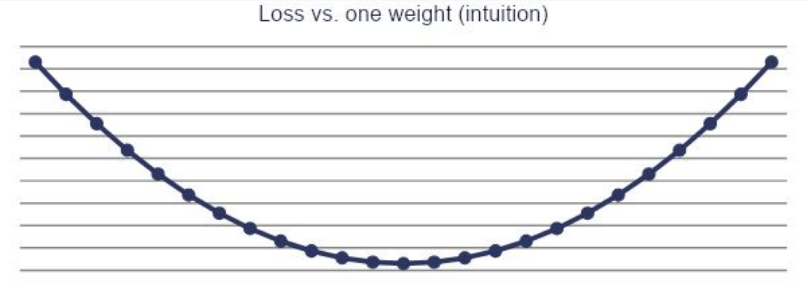
Loss functions: how 'wrong' is a prediction?

Cross-Entropy Loss

Used for classification. Penalizes confident wrong answers heavily.

Mean Squared Error (MSE)

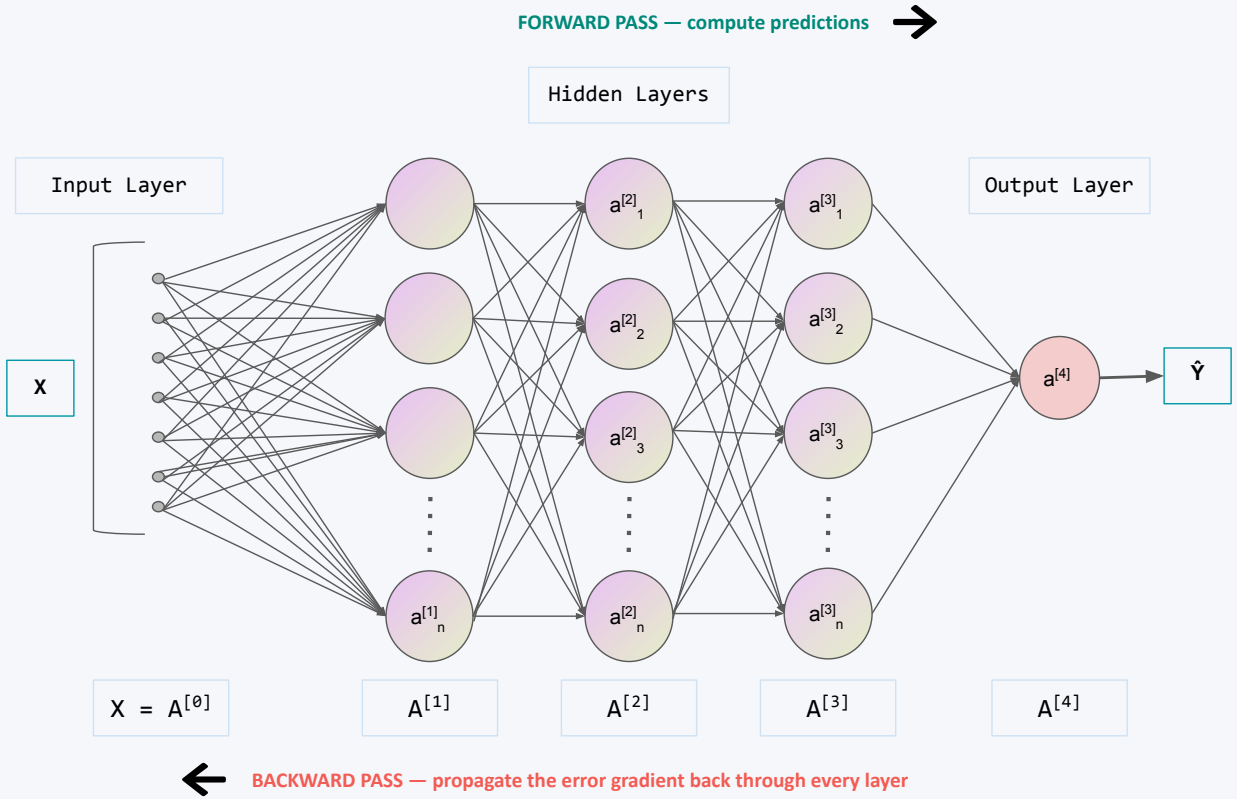
Used for regression-style outputs (e.g. predicting coordinates).



Gradient Descent: the 'rolling downhill' idea

- Imagine the loss as a hilly landscape and the network's weights as your position on it.
- The gradient points uphill (direction of steepest increase) — so we step in the opposite direction.
- The learning rate controls how big each step is. Too small = painfully slow. Too large = overshoot and bounce around.
- We repeat this for every weight, simultaneously, every training step.

Backpropagation: Assigning Blame, Layer by Layer



Optimizers: Different Ways to Take the Downhill Step

X-axis: training iterations · Y-axis: loss (lower is better)

SGD

Takes a step purely along the current gradient. Simple but can zig-zag and converge slowly.

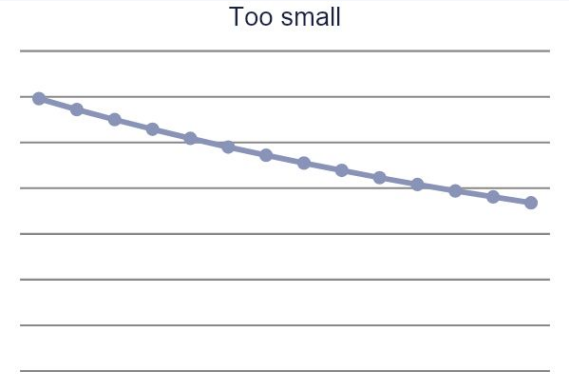
SGD + Momentum

Keeps a 'velocity' from previous steps, smoothing out zig-zags like a rolling ball.

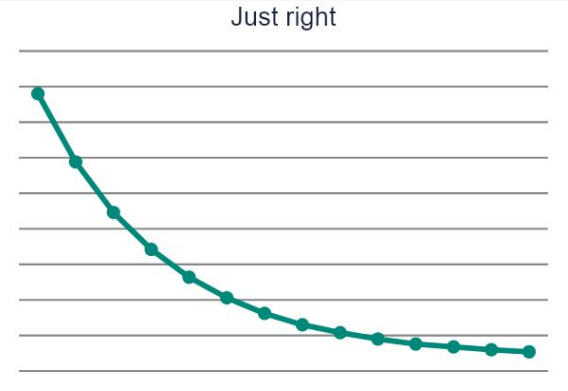
Adam

Adapts the step size per-weight using running averages of past gradients — fast, popular default.

Learning Rate: The Most Important Knob



Loss barely decreases — training would need far more time/iterations.



Loss decreases smoothly and settles near the minimum.

Steps overshoot the minimum — loss oscillates or diverges.

Batch Size, Iterations & Epochs — A Worked Example

Batch size

Number of images processed together before one weight update.

Iteration

One weight update — i.e. one pass through one batch.

Epoch

One full pass through the entire training dataset.

Worked Example

Dataset: 10,000 training images · Batch size: 50

```
Iterations per epoch = 10,000 / 50 = 200
```

If we train for 30 epochs:

Total weight updates = 200 x 30 = 6,000

- Small batches: noisier gradient estimates, can act like mild regularization, lower memory use.
- Large batches: smoother, more accurate gradient estimates, but higher memory cost and can generalize slightly worse without tuning.

SECTION 2.3

Regularization & Hyperparameter Tuning

How we stop a network from memorizing the training set instead of learning to generalize.

Why Models Overfit: Train vs. Validation Loss

Notice: after ~epoch 8 the validation loss climbs back up while training loss keeps falling — the model is starting to memorize training examples instead of learning generalizable patterns.

Underfitting (high bias)

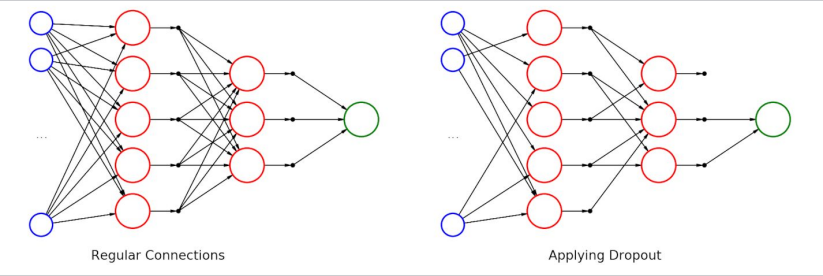
Model is too simple or undertrained — both training and validation loss stay high.

Overfitting (high variance)

Model memorizes training data's noise — training loss is very low but validation loss is high.

Dropout & Batch Normalization

Dropout



During each training step, a random subset of neurons is temporarily switched off. This forces the network to not rely too heavily on any single neuron, improving generalization. At test time, all neurons are active again.

Batch Normalization

- Layer inputs can drift to very different scales as training progresses, slowing learning down.
- Batch norm re-centers and re-scales each mini-batch's activations to have a stable mean and variance before the next layer sees them.
- Effect: training is faster and more stable, allows higher learning rates, and adds a mild regularizing effect.
- Used in nearly every modern CNN architecture .

```
normalize -> scale -> shift (per mini-batch, per layer)
```

Data Augmentation: More Training Variety for Free

Instead of collecting more images, we create realistic variations of the ones we already have — the label stays the same, but the pixels change.

Horizontal Flip

Random Crop

Rotation

Color Jitter

Random Noise

Scale / Zoom

Early Stopping & Weight Penalties

Save the model checkpoint where validation loss is lowest, then stop training even though training loss could keep falling.

Weight Penalties (L1 / L2)

- Add a penalty to the loss for having large weights — the network is discouraged from relying too heavily on any single feature.
- L2 (weight decay): penalizes the sum of squared weights — encourages small, spread-out weights. Most common in CNNs.
- L1: penalizes the sum of absolute weights — tends to push some weights to exactly zero, creating sparsity.

```
Loss_total = Loss_data + lambda x penalty(weights)
```

Finding Good Hyperparameters, and a Practical Checklist

Search strategies

Grid Search

Try every combination of a fixed value list per hyperparameter. Thorough but expensive as dimensions grow.

Random Search

Sample combinations randomly. Often finds good settings faster than grid search in high dimensions.

Automated Tuning

Methods like Bayesian optimization use past results to intelligently choose the next combination to try.

If your model overfits, try (in order):

- Add or increase data augmentation
- Add dropout, or increase the dropout rate
- Add L2 weight decay
- Reduce model size / capacity
- Use early stopping
- Get more training data, if possible

SECTION 2.4

Domain Adaptation & Transfer Learning

How we reuse what one network already learned to solve a new, smaller problem.

The Core Idea: Don't Start From Zero

An everyday analogy

A chef who has mastered Italian cooking already understands heat control, knife skills, seasoning balance, and timing. Learning French cuisine next takes far less effort than starting from a complete beginner — most of the underlying skill transfers directly.

A CNN pretrained on millions of general images (e.g. ImageNet) has already learned to detect edges, textures, shapes, and object parts. Reusing that knowledge for a new, smaller dataset — medical scans, crops, satellite images — needs far less data and time than training from scratch.

Why pretrained backbones generalize

Early layers
edges, colors, simple textures

Middle layers
shapes, patterns, object parts

Late layers
whole-object, task-specific concepts

Early/middle layers are generic — edges and textures look similar whether the image is a cat, a tumor, or a crop leaf. This is exactly why we can reuse them.

Two Ways to Reuse a Pretrained Network

Feature Extraction

FREEZE THE BACKBONE

Conv layers (frozen)

New classifier head (trained)

Keep all pretrained convolutional layers fixed (frozen) and only train a new final classifier layer on top. Fast, needs little data, lower overfitting risk.

Fine-Tuning

UNFREEZE AND RETRAIN

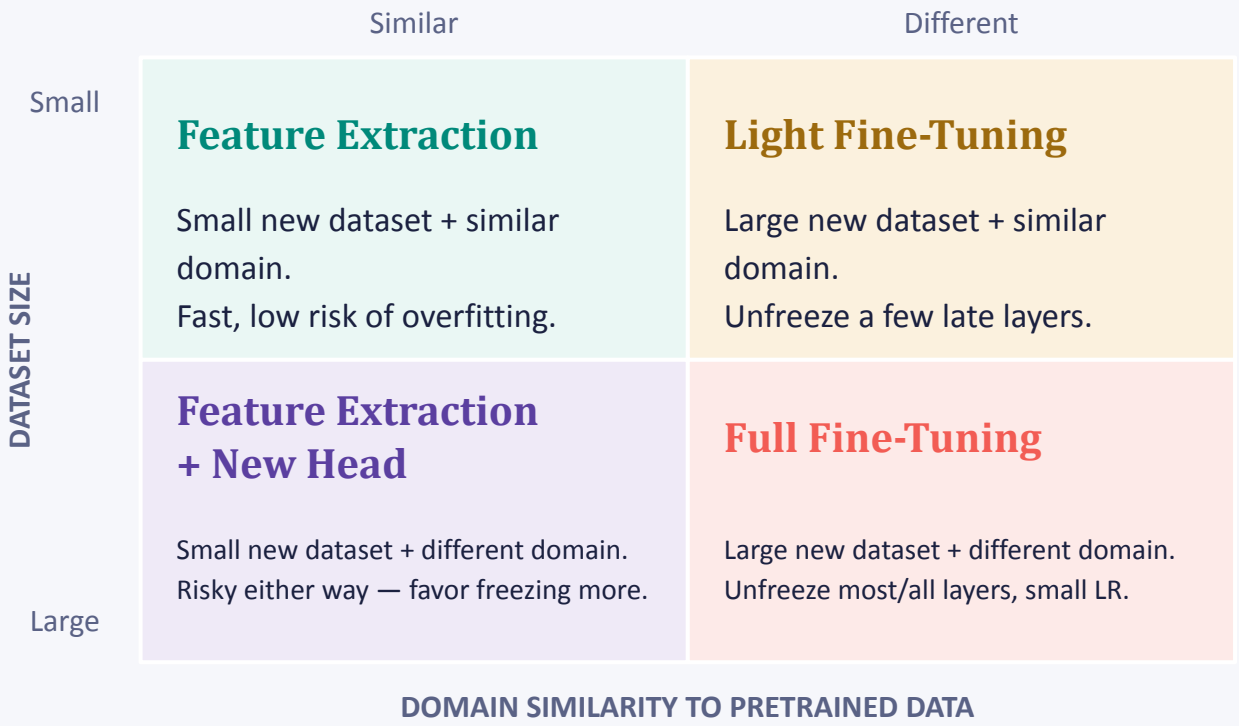
Early conv layers (frozen)

Later conv layers (trainable)

New classifier head (trained)

Unfreeze some (often later) layers and continue training them, usually with a small learning rate. Adapts more to the new domain but needs more data to avoid overfitting.

Choosing a Strategy: A Quick Decision Guide



CASE STUDY

Animal Classification with a Fine-Tuned CNN

Dataset & Challenges

Limited labeled images per species; variation in pose, lighting, background.

→

Pretrained Backbone

Start from an ImageNet-pretrained CNN (general edges/textures/shapes already learned).

→

Fine-Tune + Augment

Unfreeze later layers, train with augmentation (flips, crops, color jitter) to reduce overfitting.

→

Evaluate

Check accuracy and a confusion matrix to see which species get confused with each other.

Discussion: what would change if this pipeline classified tumors instead of animals?

Think about: dataset size, cost of a false negative vs. false positive, and which augmentations are still safe to use.

CHAPTER SUMMARY

The Full Journey: Raw Image to Fine-Tuned Model

- CNNs replace hand-built filters with learned ones using local, shared weights.
- Training is a repeated loop: predict, measure error, backpropagate, update.
- Regularization and transfer learning make models reliable with limited data.

REFERENCE

Key Terms Glossary

Kernel / Filter

Small matrix of learnable weights that slides across an image to detect a pattern.

Feature Map

Output of applying a filter across an image; highlights where a pattern was found.

Stride

Number of pixels the filter moves between applications.

Pooling

Fixed (non-learned) downsampling operation, e.g. max-pooling.

Epoch

One full pass through the entire training dataset.

Batch

A small group of training examples processed together before one weight update.

Loss Function

Measures how wrong the model's predictions are; training tries to minimize it.

Backpropagation

Algorithm that computes how much each weight contributed to the loss, layer by layer.

Optimizer

Rule for updating weights using gradients, e.g. SGD, Adam.

Overfitting

Model fits training data (incl. its noise) too closely and fails to generalize.

Dropout

Regularization that randomly disables neurons during training.

Transfer Learning

Reusing a model pretrained on one dataset/task as a starting point for another.

WRAP-UP

Discuss & Reflect

1. Why does weight sharing make CNNs far more parameter-efficient than fully-connected networks?

2. Walk through the four steps of the training loop using your own words — what does each step actually compute?

3. Your validation loss is rising while training loss keeps falling. Name two techniques from this chapter that could help, and explain why.

4. You have 300 labeled X-ray images. Would you train a CNN from scratch, do feature extraction, or fine-tune? Justify your choice using the decision guide.

Sources

- https://python.plainenglish.io/decoding-cnns-a-beginners-guide-to-convolutional-neural-networks-and-their-applications-1a8806cbf536
- https://ujjwalkarn.me/2016/08/11/intuitive-explanation-convnets/
- https://mlnotebook.github.io/post/CNN1/?zarsrc=31&gidzl=zO9s9P3ZRopmdNGYaVrj8_EQSIkChcuaxiSdA8ZoDIAwatajtF4n9UcGA7cDf3fmwCOdUMHHK2evbUTgB0
- https://adamharley.com/nn_vis/cnn/2d.html
- https://cs231n.github.io/convolutional-networks/